

```
# CampusConnect – 6-Week Implementation Plan (Independent Module Ownership)

**Team Size:** 3 Developers
**Architecture:** ASP.NET Core MVC + EF Core + SQL Server + Razor Views
**Development Model:** Full Module Ownership – no cross-person dependency
**Duration:** 6 Weeks

**Core principle of this version:** each person owns one complete, self-contained module – their own database entities, controllers, services, and views – from Week 1 to Week 5. Nobody has to wait on a teammate's feature to build or test their own. The only place modules touch each other is the final integration in Week 6 (shared dashboards/notifications, which only read finished data).

---

# 1. Module Ownership

## 👤 Person 1 – Identity, Profiles & Recommendations
**Owns end-to-end:** FR-1 (Auth/RBAC), FR-2 (Student/Faculty Profile & Skills), FR-5 (Matching/Skill Gap)

- Entities: `users`, `departments`, `student_profiles`, `faculty_profiles`, `skills`, `student_skills`
- Pages: `/Account/Login`, `/Account/Register`, `/Profile/Edit`, `/Dashboard/Student` (own version), matching components
- Why it's independent: auth is foundational and built first; profile/skills only reference the user's own data. The matching engine is built and tested against seeded dummy opportunities (Person 1 creates their own test fixtures), so it never has to wait on Person 2's real opportunity data.

## 👤 Person 2 – Opportunities, Applications & Discovery
**Owns end-to-end:** FR-3 (Opportunity/Event/Internship Management), FR-4 (Discovery & Applications), Organizer Dashboard

- Entities: `opportunities`, `applications`
```

```
- Pages: `/Opportunities/Create`, `/Admin/Opportunities`,  
`/Opportunities/Index`, `/Opportunities/Details/{id}`,  
`/Organizer/Dashboard`  
- Why it's independent: opportunity CRUD, approval workflow,  
search/filter, and applications are all self-contained within this module.  
Person 2 uses **seeded dummy users/skills** to build and test discovery  
and applications, so this module never blocks on Person 1's profile work  
being finished.
```

```
## 👤 Person 3 – Campus Services & Engagement
```

```
**Owns end-to-end:** FR-7 (Grievances), FR-8 (Directory & Networking),  
FR-9 (Faculty & Academic Interaction)
```

```
- Entities: `grievances`, `grievance_logs`, `connections`, `messages`,  
`groups`, `group_members`, `faculty_office_hours`, `announcements`  
- Pages: `/Grievances/Create`, `/Grievances/Tracker`, `/Directory/Index`,  
`/Faculty/OfficeHours`  
- Why it's independent: grievances, connections, office hours, and  
announcements only need a generic logged-in user ID – no real dependency  
on how Person 1 or 2 built their modules.
```

```
## 🍷 The one shared piece: FR-10 (Dashboards, Calendar, Notifications,  
Analytics)
```

```
This is deliberately **not** built weekly alongside the other modules –  
it's a **Week 6 integration layer only**, where each person exposes a  
simple read-only query/view-model from their own module (e.g.  
"GetUpcomingDeadlinesForUser", "GetOpenGrievanceCountForUser"). Nobody  
edits another person's tables or logic; they just consume a small  
read-only interface the module owner defines. This keeps the first 5 weeks  
fully parallel.
```

```
---
```

2. Ground Rules for Independence

```
* [ ] Each person defines and owns their own DB entities – no one edits  
another person's tables.
```

```
* [ ] Cross-module data needed for building/testing (e.g. "a user" for  
grievances, "a skill" for matching) is **seeded locally** by each person,  
not fetched from a teammate's half-built feature.
```

```
* [ ] The only real hard dependency is the shared `users`/Identity table –
Person 1 finishes this in Week 1, after which it's a stable foundation
nobody touches again. Everyone else just references `UserId` as a foreign
key.
```

```
* [ ] Integration between modules (dashboards, notifications, calendar)
happens **only in Week 6**, through small read-only interfaces each owner
exposes – never by one person reaching into another's controller/service
mid-build.
```

```
* [ ] Weekly "integration checkpoints" from the old plan are replaced with
**independent weekly deliverables** – each person can demo their own
module without the others being done.
```

```
# 3. Development Architecture (same for all three)
```

```
```text
```

```
Razor View → Controller → Service → EF Core → SQL Server
```

```
```
```

```
* [ ] Controllers: request/response only
* [ ] Business logic in Services
* [ ] ViewModels used where appropriate
* [ ] No raw SQL unless documented
* [ ] Dependency Injection used
* [ ] Async DB operations
* [ ] Authorization on protected actions
* [ ] Input validation
```

```
# WEEK 1 – Foundations (built once, then stable)
```

```
## 👤 Person 1 – Identity & Auth (foundation for everyone, built solo)
```

```
* [ ] Configure ASP.NET Core Identity
* [ ] User model, college email validation
* [ ] Registration / Login / Logout
* [ ] Role assignment (Student, Faculty, Club Coordinator, Event
Organizer, HOD, Admin)
* [ ] Role-based authorization
```

```
* [ ] Deliver stable `users` table + role scaffold others can reference by
Week 1 end
```

```
## 👤 Person 2 – Own Database Foundation (independent)
```

```
* [ ] Configure own `DbContext` section for `opportunities` /
`applications`
* [ ] Define entities, relationships, migrations
* [ ] Seed dummy users/skills for local testing (does NOT wait for Person
1)
* [ ] Seed sample opportunities/categories
```

```
## 👤 Person 3 – Own Database Foundation (independent)
```

```
* [ ] Configure own `DbContext` section for grievances/directory/faculty
entities
* [ ] Define entities, relationships, migrations
* [ ] Seed dummy users for local testing
* [ ] Shared UI shell (`_Layout`, navbar, footer, form/table/card
components) – built once, used by all
```

```
### ✅ Week 1 Deliverable (independent per person)
```

```
* Person 1: working registration/login/roles
* Person 2: opportunity entities + migrations running standalone with seed
data
* Person 3: grievance/directory entities + migrations running standalone
with seed data, shared layout ready
```

```
---
```

```
# WEEK 2
```

```
## 👤 Person 1 – Student Profile & Skills
```

```
* [ ] Profile creation/editing (department, batch, bio, links, resume)
* [ ] Skills selection with proficiency levels
* [ ] Profile completion % calculation
* [ ] Testing: own-profile-only editing, skill save/load
```

```
## 👤 Person 2 – Opportunity Creation & Approval
```

```
* [ ] Opportunity model + create form (category, description, eligibility,
stipend, deadline, capacity)
* [ ] Approval workflow: Pending → Admin/HOD → Approve/Reject → Published
```

```
* [ ] Testing entirely against seeded users, no dependency on Person 1's
profile UI

## 👤 Person 3 – Grievance Submission
* [ ] Grievance creation form (category, description, priority, anonymous
option, routing)
* [ ] Ticket creation + SLA due time calculation
* [ ] Testing entirely against seeded users

### ✅ Week 2 Deliverable (independent per person)
Each module demoable on its own with seed data – no waiting on each other.

---

# WEEK 3

## 👤 Person 1 – Matching & Recommendation Engine
* [ ] Compatibility calculation (student skills vs required skills)
* [ ] Match %, missing-skills display
* [ ] Tested against **seeded dummy opportunities** Person 1 creates – not
Person 2's live data
* [ ] Design the read-only interface Person 2 will plug real opportunity
data into later (Week 6)

## 👤 Person 2 – Discovery & Applications
* [ ] Search/filter/sort/pagination for opportunities
* [ ] Opportunity details page
* [ ] Apply / save / bookmark, application status workflow
* [ ] Organizer dashboard – applicant list, status updates, CSV export
* [ ] All tested against seeded users – no dependency on Person 1's
profile system

## 👤 Person 3 – Grievance Tracking & Audit
* [ ] Grievance workflow: Submitted → Acknowledged → In Progress →
Resolved / Escalated
* [ ] `grievance_logs` audit trail
* [ ] Student tracker, officer view, HOD escalation view

### ✅ Week 3 Deliverable (independent per person)
```

Each person has a fully working, demoable module in isolation. No shared end-to-end test yet – that only happens in Week 6.

WEEK 4

👤 Person 1 – Faculty Profiles & Dashboard Foundation

- * [] Faculty profile management
- * [] Student dashboard shell (own module data only: profile completion, skill gaps)

👤 Person 2 – Applications Depth

- * [] Full application history for a student
- * [] Duplicate-application prevention
- * [] Organizer remarks, bulk status updates

👤 Person 3 – Directory, Connections & Faculty Interaction

- * [] Student/faculty directory with filters
- * [] Connection requests (send/accept/decline), messaging
- * [] Faculty office hours (create/edit/delete slots, booking, double-booking prevention)
- * [] Announcements (faculty/HOD, department-specific or college-wide)

✅ Week 4 Deliverable

All three modules feature-complete on their own.

WEEK 5 – Each person finishes and polishes their own module

👤 Person 1

- * [] Recommendation ranking refinement
- * [] Own dashboard widgets fully working against own data

👤 Person 2

- * [] Calendar data source for opportunity deadlines (exposed as a read-only feed, not built into shared calendar yet)
- * [] Organizer/HOD-facing stats for opportunities

```
## 👤 Person 3
* [ ] Notification triggers **within their own module** (grievance
updated, new announcement, connection request, new message) – stored, not
yet wired to a shared bell/UI
* [ ] Email templates for grievance/announcement events

### ✅ Week 5 Deliverable
Each module is fully complete and independently testable. No cross-module
UI yet – that's Week 6 only.

---

# WEEK 6 – Integration, Testing, Security & Deployment

This is the **only** week where modules connect to each other, and only
through small read-only interfaces:

* [ ] Person 1 wires real opportunity data (from Person 2) into the
matching engine, replacing seeded fixtures
* [ ] Person 2 exposes a read-only "upcoming deadlines" feed
* [ ] Person 3 exposes a read-only "grievance status" and "announcements"
feed
* [ ] Shared unified calendar/dashboard/notification bell pulls from these
three read-only feeds
* [ ] Admin/HOD dashboards aggregate stats from all three modules
(read-only)

## Security & RBAC (all three, on their own module)
* [ ] Each person tests authorization on their own pages (own module only)
* [ ] Anonymous grievance data protected (Person 3)
* [ ] Cross-role access checks (e.g. student can't approve opportunities)

## Performance
* [ ] Each person optimizes queries in their own module (pagination,
async, avoid N+1)
* [ ] 2-second response target verified per module

## Deployment
* [ ] Production config, connection strings, secrets
* [ ] Migrations applied for all three modules together
```

```

* [ ] Azure SQL + Azure App Service deployment
* [ ] Full end-to-end test across all modules

---

# 4. Git Workflow

```text
main
└─ develop
 ├── feature/identity-profile (Person 1)
 ├── feature/opportunities (Person 2)
 └─ feature/campus-services (Person 3)
```

* [ ] Each person works almost entirely within their own feature
branch/folder – minimal merge conflicts since entities/controllers/views
don't overlap
* [ ] PRs reviewed by teammates for code quality, but reviewers are **not
blockers** for the owner's own testing/demo
* [ ] Only Week 6 integration branch touches more than one module's code

---

# 5. Definition of Done (per module, per person)

* [ ] Entities + migrations complete
* [ ] Service + Controller + ViewModel + Razor View implemented
* [ ] Validation + Authorization implemented
* [ ] Error handling + success/failure messaging
* [ ] Tested with valid/invalid/unauthorized input
* [ ] Demoable standalone, without needing the other two modules finished
* [ ] Code reviewed and merged to `develop`

---

# 6. Six-Week Milestone Tracker (per person, independent)

Week	Person 1	Person 2	Person 3

```

```
| 1 | Auth + roles live | Opportunity DB + seed data | Grievance/Directory  
DB + seed data, shared UI shell |  
| 2 | Profile + skills | Opportunity creation + approval | Grievance  
submission |  
| 3 | Matching engine (seeded data) | Discovery + applications + organizer  
dashboard | Grievance tracking + audit trail |  
| 4 | Faculty profiles + own dashboard shell | Application depth + remarks  
| Directory, connections, office hours, announcements |  
| 5 | Recommendation refinement | Deadline feed exposed | Notification  
triggers + email templates |  
| 6 | **Integration**: real data wired into matching, security, deployment  
|
```

★ Key difference from the old plan

No week requires Person A to wait for Person B's feature to be done. Every person can build, test, and demo their entire module start-to-finish without the other two existing yet. The three modules only meet each other once, in Week 6, through small read-only feeds – not through shared mid-build logic.